

114 學年度第一學期 資料庫管理系統 (LIS5076)

期末專題書面報告

組別：第二組

主題：動漫目錄與追蹤系統

B12106002	圖資三	王苡寧
B12106015	圖資三	陳歆雅
B12106020	圖資三	黃聿瑄
B12106054	圖資三	詹凱昕
B12302315	政治三	吳青樺

壹、 背景介紹

一、研究動機與主題選擇原因

近年來日本動漫產業快速成長，作品數量、類型與跨媒體改編日益增加，觀眾在追蹤、管理與回顧觀看紀錄時，常面臨資訊分散、紀錄不完整或社群互動零散等問題，目前現有的動漫平台多半著重於播放或單一功能（如評分或留言），較少能同時整合「作品資料管理」、「觀看進度追蹤」與「社群互動」三大核心需求。

因此，本專題選擇「動漫目錄與追蹤系統」作為研究主題，期望透過資料庫管理系統的設計，建構一個結構化、可擴充且具一致性規則的系統，讓使用者能有效管理動漫資訊、追蹤觀看狀態，並與其他粉絲進行互動與交流。

二、資料庫管理系統在本主題中的功能與影響

(一) 資料一致性與完整性控管

透過主鍵、外鍵與限制條件等控制，確保動漫、季度、劇集、角色、用戶等資料之間關聯正確無誤，避免資料重複或產生資料矛盾。

(二) 複雜關係的有效管理

本系統涉及多對多關係（如：用戶追蹤、動漫與類型、角色與聲優），資料庫能透過關聯表清楚表達及管理這些關係。

(三) 支援動態計算與追蹤機制

系統可即時計算動漫平均評分（average_rating）、評分人數（rating_count），並追蹤使用者的觀看進度與完成狀態，提供即時資訊分析。

(四) 提升系統擴充性與維護

良好的資料庫結構可支援未來新增功能（如多語言配音、跨平台資料整合），降低系統維護成本。

三、資料庫所存放的內容

(一) 使用者資料：帳號、電子郵件、追蹤關係、觀看狀態

(二) 動漫基本資料：作品名稱、製作公司、類型、目標觀眾、改編來源

(三) 內容結構資料：季度（Seasons）、劇集（Episodes）

(四) 互動資料：評分（Ratings）、評論（Reviews）、回覆（Comments）、追蹤

(五) 角色與製作人員資料：角色設定、聲優、語言、製作團隊職位

(六) 主題曲資料：開場／結尾歌曲及其適用劇集區間

這些資料彼此高度關聯，需透過資料庫管理系統進行系統化整合。

四、實際用途

(一) 建立完整且具搜尋功能的動漫目錄

- (二) 協助使用者追蹤觀看進度與更新紀錄
- (三) 提供評分與評論機制，促進使用者間的交流互動
- (四) 支援角色、聲優與製作人員的資料查詢
- (五) 作為未來推薦系統或使用者行為分析的資料基礎

五、預期幫助

對使用者而言，本系統提供一個集中式的平台，協助有系統地管理動漫觀看紀錄、評價作品並與社群互動，提升整體使用體驗。

對系統管理者或內容平台而言，透過結構良好的資料庫設計，可有效管理大量動漫與使用者資料，並為後續資料分析、推薦機制或商業應用奠定基礎。

貳、 Business Rules 說明

一、用戶與社群互動規則 (User & Social Interaction)

- (一) 帳戶唯一性：每位用戶必須擁有唯一的 username 與 email。
- (二) 社群追蹤：用戶可以追蹤其他用戶（多對多關係）。
 - 1. 追蹤關係為單向，用戶 A 追蹤用戶 B，並不代表用戶 B 自動追蹤用戶 A。
 - 2. 在 *User_Follow* 關聯實體中，*follower_user_id* 不得等於 *followed_user_id*，使用者不可追蹤自己。
- (三) 評論與回覆：
 - 1. 用戶對特定動漫只能發布一則「評論 (Review)」。
 - 2. 回覆必須記錄來源用戶，以追蹤互動情形。
 - 3. 「回覆 (Comments)」必須附屬於某個評論之下，且一則評論可以有多個回覆（一對多）。

二、動漫核心架構規則 (Anime Core Structure)

- (一) 製作歸屬
 - 1. 每部動漫必須關聯至一個主要的「製作公司 (Studio)」。
 - 2. 若需記錄多個合作製作公司，則此 1:N 關係需擴充為多對多關係（目前依規格定義為一對多）。
- (二) 分類
 - 一部動漫可以同時屬於多個「類型 (Genres)」。(兒童、少年、少女、青年、女性、成人、闔家觀看)
- (三) 受眾
 - 一部動漫可以標註多個「目標觀眾 (Demographics)」(如：同時標註為少年與青年)。
- (四) 改編來源
 - 1. 動漫可以標記其原始素材來源（如：漫畫、輕小說）。

2. 若來源為漫畫，透過 ANIME_MANGA_ADAPTATION 建立多對多連結，以記錄不同版本或重製版動漫與原作漫畫的關係。
3. `is_primary_adaptation` 用於標示主要動畫版本，對於同一部漫畫 (`manga_id`)，最多只能有一筆 `is_primary_adaptation = true`

三、內容分層與順序規則 (Content Hierarchy & Sequence)

(一) 季度與劇集控制：

1. 一部動漫可以劃分為多個「季度 (Seasons)」。
2. 「劇集 (Episodes)」必須同時關聯至特定的「動漫」與「季度」。
3. 劇集編號 (`episode_number`) 應受季度約束（例如第一季的第 1-12 集）。

(二) 主題曲關聯：

1. 主題曲必須明確區分為「開場 (Opening)」或「結尾 (Ending)」。
2. 需記錄該歌曲適用的劇集區間 (`episode_start` 至 `episode_end`)。

(三) 角色分配：

角色歸屬於特定的動漫，且必須設定其重要性等級（主角、配角、小角色）。

(四) 聲優配音邏輯：

1. 「角色」與「聲優」為多對多關係（同一角色在不同語言、不同時期可能有不同聲優）。
2. 在關聯表中必須註明「語言」（如：日語、英語），以區分各國配音版本。

(五) 製作團隊職責：

「製作人員 (Staff)」透過關聯表與「動漫」連結，且必須註明其職位 (Role)，如：導演、編劇或作曲家。

四、追蹤與評價規則 (Tracking & Rating)

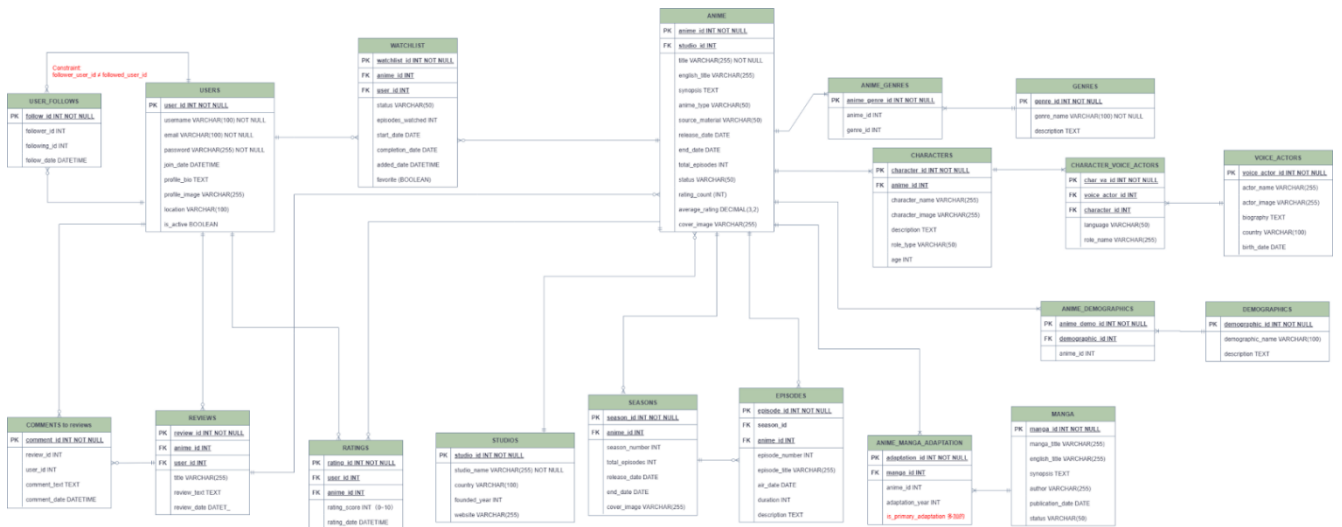
(一) 觀看狀態管理：

1. 用戶在追蹤列表中，必須選擇動漫的狀態（正在看、已完成、計劃觀看）。
2. `episodes_watched`（已看集數）不得超過該動漫或季度的 `total_episodes`。
3. 當狀態變更為「已完成」時，應記錄 `completion_date`。

(二) 評分機制：

1. 系統應區分「評分 (Ratings)」與「評論 (Reviews)」。
2. 動漫的 `average_rating` 應根據所有用戶在 RATINGS 表中的 `rating_score` 自動計算更新。
3. `rating_count` 應同步記錄參與評分的總人數。

五、Entity-Relationship Diagram 實體關聯圖



參、SQL (Structured Query Language)指令

一、簡單 Query

(一) 列出每位漫畫作者，以及他們的所有作品，並且同時顯示這些作品是否有動畫改編及改編的動畫名稱

```
1 SELECT M.author,
2        M.manga_title,
3        A.title AS anime_title,
4        ama.adaptation_year
5 FROM MANGA M
6 LEFT JOIN ANIME_MANGA_ADAPTATION ama ON M.manga_id = ama.manga_id
7 LEFT JOIN ANIME A ON ama.anime_id = A.anime_id
8 ORDER BY M.author, M.manga_title, ama.adaptation_year;
```

	author	manga_title	anime_title	adaptation_year
	吾峠呼世晴	鬼滅之刃	鬼滅之刃	2019
	山田鐘人	葬送的芙莉蓮	葬送的芙莉蓮	2023
	芥見下々	咒術迴戰	咒術迴戰	2020
▶	諫山創	進擊的巨人	進擊的巨人 Final Season	2020

(二) 查詢每位使用者追蹤的人數

```
1 SELECT u.username,  
2 COUNT(f.following_id) AS following_count  
3 FROM USERS u  
4 LEFT JOIN USER_FOLLOWS f  
5     ON u.user_id = f.follower_id  
6 GROUP BY u.user_id, u.username  
7 ORDER BY following_count DESC;
```

	username	following_count
	AnimeLover99	2
►	CyberRunner	1
	IsekaiTrash	1
	LazyCat_Watching	1
	NoAnimeNoLife	1
	SakuraWatcher	1
	DarkAnimeLover	0
	DetectiveConan	0
	FantasyAddict	0
	GhibliFan	0
	HorrorSeeker	0
	MangaReader_X	0
	MechaKing	0

(三) 已完成觀看的動漫清單

```
1 SELECT
2     u.username,
3     A.title AS anime_title,
4     w.completion_date
5 FROM WATCHLIST w
6 JOIN USERS u ON w.user_id = u.user_id
7 JOIN ANIME A ON w.anime_id = A.anime_id
8 WHERE w.status = '已完成'
9 ORDER BY w.completion_date DESC;
```

	username	anime_title	completion_date
▶	SportsSpirit	排球少年！！	2024-01-20
	NoAnimeNoLife	轉送的芙莉蓮	2023-12-25
	MusicAnimeLover	孤獨搖滾！	2023-12-15
	SakuraWatcher	間諜家家酒	2023-05-20
	MangaReader_X	鋼之鍊金術師 BROTHERHOOD	2023-03-15
	AnimeLover99	咒術迴戰	2023-02-15
	Reviewer_Pro	咒術迴戰	2023-01-20

二、中等 Query

(一) 平均評分高於整體八成的動漫

```
1 -- 1.算出每部動漫的排名、總數、以及全站平均分數，為後續門檻計算做準備
2 WITH ranked AS (
3     SELECT
4         title,
5         average_rating,
6         rating_count,
7         ROW_NUMBER() OVER (ORDER BY rating_count) AS rn,
8         COUNT(*) OVER () AS total_cnt,
9         AVG(average_rating) OVER () AS avg_rating
10    FROM ANIME
11
12
13 -- 2.從排好序的名單中，精確抓出第 80% 位置的作品，提取它的「評分人數」作為及格線
14
15 threshold AS (
16     SELECT rating_count AS rating_80th, avg_rating
17    FROM ranked
18   WHERE rn = GREATEST(1, FLOOR(total_cnt * 0.8) + 1)
19 )
20
21 --3. 將所有作品與 PR80 門檻進行比對，過濾出「高分」且「高人氣」的贏家
22 SELECT r.title AS anime_title,
23        r.average_rating,
24        r.rating_count
25 FROM ranked r
26 CROSS JOIN threshold t
27 WHERE r.average_rating > t.avg_rating
28    AND r.rating_count >= t.rating_80th
29 ORDER BY r.average_rating DESC, r.rating_count DESC;
```

Result Grid			
Filter Rows:			
Export:  Wrap C			
	anime_title	average_rating	rating_count
	鋼之鍊金術師 BROTHERHOOD	9.10	3200000
	獵人 (2011)	9.00	2600000
	進擊的巨人 Final Season	9.00	1800000
▶	咒術迴戰	8.70	1250000

(二) 已知某部動漫作品中的某個聲優，查找該聲優其他參與的作品：想要找進擊的巨人，角色里維·阿卡曼的聲優以及他有參與過的其他作品

```

1  -- 1. 先找出《進擊的巨人》中「里維·阿卡曼」的聲優 ID
2  WITH TargetVA AS (
3      SELECT
4          va.voice_actor_id,
5          va.actor_name
6      FROM VOICE_ACTORS va
7      JOIN CHARACTER_VOICE_ACTORS cva ON va.voice_actor_id = cva.voice_actor_id
8      JOIN CHARACTERS C ON cva.character_id = C.character_id
9      JOIN ANIME A ON C.anime_id = A.anime_id
10     WHERE A.title LIKE '%進擊的巨人%'
11           AND C.character_name LIKE '%里維%'
12 )
13 -- 2. 根據聲優 ID，列出他參與過的所有動漫作品與對應角色
14 SELECT
15     tva.actor_name AS '聲優姓名',
16     A.title AS '動漫作品',
17     C.character_name AS '飾演角色',
18     C.role_type AS '角色定位'
19 FROM TargetVA tva
20 JOIN CHARACTER_VOICE_ACTORS cva ON tva.voice_actor_id = cva.voice_actor_id
21 JOIN CHARACTERS C ON cva.character_id = C.character_id
22 JOIN ANIME A ON C.anime_id = A.anime_id
23 ORDER BY A.release_date DESC;

```

-- 1.

	聲優 姓名	動漫作品	飾演角色	角色 定位
▶	神谷浩史	進擊的巨人 Final Season	里維·阿卡曼	配角
	神谷浩史	文豪野犬	江戸川亂步	配角
	神谷浩史	排球少年！！	武田一鐵	配角

(三) 列出每部動畫的名稱，並透過子查詢即時計算該動畫在 `characters` 表格中擁有多少位角色。

```

1 SELECT
2     A.title,
3     A.anime_type,
4     (
5         /* 相關子查詢：根據 anime_id 計算該動畫的角色數量 */
6         SELECT COUNT(*)
7         FROM CHARACTERS C
8         WHERE C.anime_id = A.anime_id
9     ) AS character_count
10 FROM anime A
11 ORDER BY character_count DESC;

```

	title	anime_type	character_count
	葬送的芙莉蓮	電視	1
	孤獨搖滾！	電視	1
	靈能百分百	電視	1
	文豪野犬	電視	1
	鏈鋸人	電視	1
	藍色監獄	電視	1
	藥師少女的獨語	電視	1
	排球少年！！	電視	1
	鋼之鍊金術師 BROTHE...	電視	0
	獵人 (2011)	電視	0
	國王排名	電視	0
	物理魔法使馬修	電視	0
	第一神拳	電視	0
	鬼滅之刃 劇場版 無限...	電影	0
	咒術迴戰 0	電影	0
	我推的孩子 第一季	NULL	0
	BEASTARS 第一季	NULL	0
	我推的孩子 第一季	NULL	0
	BEASTARS 第一季	NULL	0

三、進階 Query

(一) 評分高但評論很少的冷門神作

```
1 WITH ReviewStats AS (  
2     -- [CTE 1] 算每部動漫的評論總數 (處理沒有評論的動漫，所以用 LEFT JOIN)  
3     SELECT  
4         A.anime_id,  
5         COUNT(r.review_id) AS review_count  
6     FROM ANIME A  
7     LEFT JOIN REVIEWS r ON A.anime_id = r.anime_id  
8     GROUP BY A.anime_id  
9 ),
```

```
10 RatingStats AS (  
11     -- [CTE 2] 算每部動漫的總評分人數 (確保高分不是只有 1 個人投出來的)  
12     SELECT  
13         anime_id,  
14         COUNT(rating_id) AS vote_count  
15     FROM RATINGS  
16     GROUP BY anime_id  
17 )
```

```
18 SELECT  
19     A.title AS '作品名稱',  
20     A.average_rating AS '平均評分',  
21     rs.review_count AS '評論數量',  
22     rt.vote_count AS '評分人數'  
23 FROM ANIME A  
24 JOIN ReviewStats rs ON A.anime_id = rs.anime_id  
25 JOIN RatingStats rt ON A.anime_id = rt.anime_id
```

```
26 WHERE  
27     -- 1. 評分門檻：高於全站平均分 (例如 8.0 以上)  
28     A.average_rating > (SELECT AVG(average_rating) FROM ANIME)  
29  
30     -- 2. 冷門門檻：評論數低於或等於全站平均評論數  
31     AND rs.review_count <= (SELECT AVG(review_count) FROM ReviewStats)  
32  
33     -- 3. 可信度門檻：至少有 2 個人評過分 (避免只有 1 人給 10 分的偏誤)  
34     AND rt.vote_count >= 2  
35  
36 ORDER BY A.average_rating DESC, rt.vote_count ASC;
```

	作品名稱	平均 評分	評論 數量	評分 人數
▶	第一神拳	9.67	0	3
	鋼之鍊金術師 BROTHERHOOD	9.10	1	4
	獵人 (2011)	9.00	0	2
	我推的孩子	8.80	0	2
	藥師少女的獨語	8.70	1	2

(二) 找出「觀看清單高度重疊、尚未互追，且對方活躍（評論數高於系統平均且至少自己兩倍）」的使用者，提供演算法，使用者在瀏覽時會被推播「你可能有興趣的評論用戶」

```

1 WITH UserActivity AS (
2     -- [CTE 1] 計算每個人的評論總數
3     SELECT user_id, COUNT(review_id) AS review_cnt
4     FROM REVIEWS
5     GROUP BY user_id
6 ),
7 SharedWatchlist AS (
8     -- [CTE 2] 找出觀看清單有重疊的使用者對 (User A, User B)
9     -- 設定「高度重疊」為共同看過 3 部以上
10    SELECT w1.user_id AS user_a, w2.user_id AS user_b, COUNT(*) AS overlap_count
11    FROM WATCHLIST w1
12    JOIN WATCHLIST w2 ON w1.anime_id = w2.anime_id AND w1.user_id <> w2.user_id
13    GROUP BY w1.user_id, w2.user_id
14    HAVING COUNT(*) >= 3
15 )

```

```

16 SELECT
17     u_a.username AS '推薦者',
18     u_b.username AS '建議關注對象',
19     sw.overlap_count AS '共同觀看作品數',
20     ua_b.review_cnt AS '對方評論數',
21     ua_a.review_cnt AS '自己評論數'
22 FROM SharedWatchlist sw
23 JOIN USERS u_a ON sw.user_a = u_a.user_id
24 JOIN USERS u_b ON sw.user_b = u_b.user_id
25 JOIN UserActivity ua_a ON sw.user_a = ua_a.user_id
26 JOIN UserActivity ua_b ON sw.user_b = ua_b.user_id
27 WHERE
28     -- 1. 尚未互追 (A 沒追蹤 B，且 B 也沒追蹤 A)
29     NOT EXISTS (
30         SELECT 1 FROM USER_FOLLOWS f
31         WHERE (f.follower_id = sw.user_a AND f.following_id = sw.user_b)
32             OR (f.follower_id = sw.user_b AND f.following_id = sw.user_a)
33     )
34     -- 2. 對方活躍度門檻：高於全系統平均
35     AND ua_b.review_cnt > (
36         SELECT AVG(review_cnt) FROM UserActivity
37     )
38     -- 3. 對方活躍度門檻：至少是自己的兩倍
39     AND ua_b.review_cnt >= (ua_a.review_cnt * 2)
40 ORDER BY sw.overlap_count DESC, ua_b.review_cnt DESC;

```

	推薦者	建議關注對象	共同觀看作品數	對方評論數	自己評論數
▶	LazyCat_Watching	Reviewer_Pro	3	13	2

(三) 找出「對特定類型有明確偏好（平均評分最高）、且該類型中仍有高分作品尚未加入觀看清單」的使用者與作品組合。提供推薦演算法，當使用者在首頁瀏覽時，系統會精準推播：「在你最愛的類型中，還有這部錯過的經典」

```
1 WITH UserGenrePreference AS (  
2   -- [1] 分析每位使用者的偏好：計算他們對各類型的平均評分  
3   SELECT  
4     r.user_id,  
5     ag.genre_id,  
6     ROUND(AVG(r.rating_score), 2) AS user_genre_avg,  
7     -- 找出每個使用者評分最高的類型排名  
8     ROW_NUMBER() OVER (PARTITION BY r.user_id ORDER BY AVG(r.rating_score) DESC) AS preference_rank  
9   FROM RATINGS r  
10  JOIN ANIME_GENRES ag ON r.anime_id = ag.anime_id  
11  GROUP BY r.user_id, ag.genre_id  
12 ),  
13 CandidateAnime AS (  
14   -- [2] 撈出使用者「本命類型」中，他們還沒看過的作品  
15   SELECT  
16     ugp.user_id,  
17     G.genre_name AS favorite_genre,  
18     A.title AS recommended_anime,  
19     ROUND(A.average_rating, 2) AS anime_score,  
20     -- 在該類型內按作品總分排名  
21     ROW_NUMBER() OVER (PARTITION BY ugp.user_id ORDER BY A.average_rating DESC) AS anime_rank  
22   FROM UserGenrePreference ugp  
23   JOIN GENRES G ON ugp.genre_id = G.genre_id  
24   JOIN ANIME_GENRES ag ON G.genre_id = ag.genre_id  
25   JOIN ANIME A ON ag.anime_id = A.anime_id  
26   -- 排除條件：該作品不能出現在該使用者的觀看清單中  
27   WHERE ugp.preference_rank = 1 -- 只取使用者的「本命類型」  
28     AND NOT EXISTS (  
29       SELECT 1 FROM WATCHLIST w  
30       WHERE w.user_id = ugp.user_id AND w.anime_id = A.anime_id  
31     )  
32 )
```

```
33 -- [3] 最終輸出：每位使用者推薦一部最強遺珠  
34 SELECT  
35   u.username AS '使用者',  
36   ca.favorite_genre AS '本命類型',  
37   ca.recommended_anime AS '推薦作品',  
38   ca.anime_score AS '作品評分'  
39 FROM CandidateAnime ca  
40 JOIN USERS u ON ca.user_id = u.user_id  
41 WHERE ca.anime_rank = 1  
42 ORDER BY u.username;
```

Result Grid Filter Rows: Export: Wrap Cell Cont:				
	使用者	本命類型	推薦作品	作品 評分
▶	AnimeLover99	劇情	我推的孩子	8.80
	DarkAnimeLover	日常生活	韓送的芙莉蓮	9.10
	DetectiveConan	日常生活	韓送的芙莉蓮	9.10
	GhibliFan	日常生活	韓送的芙莉蓮	9.10
	IsekaiTrash	喜劇	孤獨搖滾！	8.90
	MangaReader_X	冒險	韓送的芙莉蓮	9.10
	MechaKing	冒險	韓送的芙莉蓮	9.10
	NoAnimeNoLife	日常生活	孤獨搖滾！	8.90
	OldSchool_Fan	冒險	韓送的芙莉蓮	9.10
	Reviewer_Pro	冒險	獵人 (2011)	9.00
	SakuraWatcher	音樂藝術	孤獨搖滾！	8.90
	SliceOfLife92	音樂藝術	孤獨搖滾！	8.90
	VoiceActorStalker	運動	第一神拳	9.67

肆、資料庫系統建置之整體心得感想

一、吳青樺

比起在其他學習管道主要基本上都在教後端如何寫指令做實務作業，在此堂課中，更從資料庫後端系統設定、語法邏輯開始學習，到從零開始架設一個資料庫語言查詢系統，身為非技術背景學生，第一次修習圖資系的課程，雖然在語法操作技能上尚未十分得心應手，但對於資料庫系統有了更多基礎理解，在理論與實務應用上都有充分的收穫。

在這次專案中從零開始發想一個資料庫系統的應用，能夠情境式思考課堂中所學，更清楚並具體的瞭解學習資料庫系統管理在實際工作處理與資訊整合時，能發揮的助益有多強大。我們在主題發想上，針對現有動漫平台中較少呈現使用者社群互動歷程的問題，透過資料庫系統追蹤並統整使用者相關資訊，讓我對資料庫應用情境的理解更紮實，也讓我在日常生活中開始主動思考資料庫系統可能的實際應用場景。此外，在專案進行的過程中，透過與組員的合作與討論，也學習到許多不同工具的使用方式與實作經驗，對我而言是一段相當有收穫的學習歷程。

二、王苡寧

實際製作一個資料庫比想像中複雜許多，從一開始的 business rules 和 ERD 圖繪製就開始不斷地邊做邊改，有時候我們也難以想像實際操作時的情況。在進行資料匯入和 query 發想時也會遇到測資不足所以沒有結果的問題，這也讓我發現一個真實

的 database 需要龐大的資料同時也會有更複雜的結構。但是這個專題也讓我實現了我在追動畫時希望可以有的平台和功能，很有趣。

三、詹凱昕

本次專案讓我實際體會到建構一個資料庫的每一個流程，從畫 ERD 開始，一路到建立 CREATE TABLE、測試自己想出來的 Query，剛開始上這們課的時候一直在想，為什麼不直接用 EXCEL 解決所有事情就好，還需要多學一套軟體跟語言，並且還要透過繁雜的前置作業，例如畫 ERD 跟正規化才可以實現。但在建好所有的表格，輸入第一個 Query 之後，我就改觀了，在面對龐雜並且多種類的資訊時，良好的資料庫管理系統就能派上用場。

在測試資料庫時，常常會遇到指令條件設計嚴密，以致於跑得出最終 table，但無法顯示資料的窘境，為此還需要特別依照我們所設計的指令條件塞測資，除此之外，在最後檢視每一個 Query 的時候，還會發現自己其實有一些 Entity(Table)忘了建立的情形，回想起來實在蠻好笑的。

經過了這個專案，除了讓我更了解資料庫在實務上的應用，也讓我針對課堂上學到的理論、ERD、正規化有更深一層的認識，雖然經過了整個學期 ERD 還是不太會化，但做完整個專案之後有種豁然開朗的感覺，也讓我對資料庫的各個需要注意的細節，有更進一步的認識！

四、陳歆雅

透過本次專案，我更清楚在建構資料庫時的步驟，還有如何實作，雖然我在課上聽的較模糊也較無法實際聯想，但是透過專案，從發想主題，到繪製 ERD，再到正規化跟執行 Query，讓我能逐步將抽象概念轉化為可操作的實務經驗。在專案進行過程中，組內歷經長時間的討論與協調，特別是在實作階段，經常因為資料結構，或關聯設計出現問題而需要反覆修改，透過不斷滾動式修正，以建立穩健的資料庫系統。

現在資料庫跟 SQL 領域都發展非常成熟，在畫 ERD 時，發現有多種功能強大的平台及工具，可以來協助繪製。在許多論壇、Youtube 也有許多關於 SQL 語法的討論及教學，對於自學與問題排除都有相當大的幫助，在學習上助益很大。

我們的主題為動漫目錄與追蹤系統，在報告當天發現有一組跟我們主題類似，但我們有追蹤其他用戶、評分評論的功能，多了一點用戶互動上的功能，也可以讓用戶在平台上發掘更多有興趣的作品，若引入更多測資，有發展類似演算法推薦功能的潛力。

五、黃聿瑄

在平時上課時，其實我對課堂上介紹的許多內容都一知半解，包含資料表關聯、正規化以及較複雜的 SQL 查詢語法，常常只是跟著範例看，卻不確定自己是否真的理解。一直到期末專題開始，需要實際設計一個完整的資料庫系統時，才發現如果只是聽課而沒有動手實作，很多觀念其實很難理解。

在這次專題中，許多查詢語句都是在反覆嘗試與修改中慢慢想出來的。從一開始不知道該怎麼把需求轉成 SQL 條件，到後來跟組員能夠自己設計包含多個資料表、彙總函數與子查詢的 query，這個過程雖然花了不少時間，但也讓我逐漸理解資料庫查詢背後的邏輯，而不只是單純套用語法。

這次以「動漫目錄與追蹤系統」作為專題主題，對我來說也是一個很大的動力，因為內容貼近日常生活，我也非常喜歡看動漫，讓我花時間去思考功能怎麼設計、資料怎麼放才合理。雖然一開始對資料庫並沒有太多信心，但透過實際完成這個專題，我發現自己對資料庫的理解確實比上課時進步許多。